

HERANÇA, REESCRITA E POLIMORFISMO

"O homem absurdo é aquele que nunca muda." -- Georges Clemenceau

Ao final deste capítulo, você será capaz de:

- Dizer o que é herança e quando utilizá-la;
- Reutilizar código escrito anteriormente;
- Criar classes filhas e reescrever métodos;
- Usar todo o poder que o polimorfismo oferece.

9.1 REPETINDO CÓDIGO?

Como toda empresa, nosso banco tem funcionários. Modelemos a classe `Funcionario` :

```
public class Funcionario {  
    private String nome;  
    private String cpf;  
    private double salario;  
    // métodos devem vir aqui  
}
```

Além de um funcionário comum, há também outros cargos, como os gerentes; estes guardam a mesma informação que um funcionário comum, mas também têm outros dados e funcionalidades um pouco diferentes. Por exemplo, um gerente no nosso banco tem uma senha numérica que permite o acesso ao sistema interno do banco, além do número de funcionários os quais ele gerencia:

```
public class Gerente {  
    private String nome;  
    private String cpf;  
    private double salario;  
    private int senha;  
    private int numeroDeFuncionariosGerenciados;  
  
    public boolean autentica(int senha) {  
        if (this.senha == senha) {  
            System.out.println("Acesso Permitido!");  
            return true;  
        } else {  
            System.out.println("Acesso Negado!");  
            return false;  
        }  
    }  
}
```

```

    }

    // outros métodos
}

```

PRECISAMOS MESMO DE OUTRA CLASSE?

Poderíamos ter deixado a classe `Funcionario` mais genérica, mantendo nela a senha de acesso e o número de funcionários gerenciados. Caso o funcionário não fosse um gerente, deixaríamos esses atributos vazios.

Essa é uma possibilidade, porém, com o tempo, podemos passar a ter muito atributos opcionais, e a classe ficaria estranha. E em relação aos métodos? A classe `Gerente` tem o método `autentica`, que não faz sentido existir em um funcionário o qual não é gerente.

Se tivéssemos um outro tipo de funcionário que tem características diferentes do funcionário comum, precisaríamos criar uma outra classe e copiar o código novamente!

Além disso, se um dia precisarmos adicionar uma nova informação a todos os funcionários, precisaremos passar por todas as classes de funcionário e adicionar esse atributo. O problema acontece novamente por não centralizarmos os dados principais do funcionário em um único lugar.

Existe um jeito, em Java, de relacionarmos uma classe de tal maneira que uma delas **herda** tudo que a outra tem. Isso é uma relação de classe mãe e classe filha. No nosso caso, gostaríamos de fazer com que o `Gerente` tivesse tudo que um `Funcionario` tem, gostaríamos que ela fosse uma **extensão** de `Funcionario`. Fazemos isso por meio da palavra-chave `extends`.

```

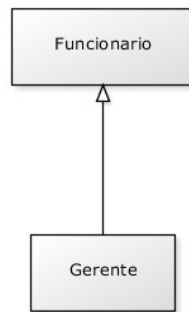
public class Gerente extends Funcionario {
    private int senha;
    private int numeroDeFuncionariosGerenciados;

    public boolean autentica(int senha) {
        if (this.senha == senha) {
            System.out.println("Acesso Permitido!");
            return true;
        } else {
            System.out.println("Acesso Negado!");
            return false;
        }
    }

    // setter da senha omitido
}

```

Em todo momento que criarmos um objeto do tipo `Gerente`, este terá também os atributos definidos na classe `Funcionario`, pois um `Gerente` **é um** `Funcionario`:



```
public class TestaGerente {
    public static void main(String[] args) {
        Gerente gerente = new Gerente();

        // podemos chamar métodos do Funcionario:
        gerente.setNome("João da Silva");

        // e também métodos do Gerente!
        gerente.setSenha(4231);
    }
}
```

Dizemos que a classe `Gerente` **herda** todos os atributos e métodos da classe mãe, no nosso caso, a `Funcionario`. Para ser mais preciso, ela também herda os atributos e métodos privados, porém não consegue acessá-los diretamente. Para acessar um membro privado na filha indiretamente, seria necessário que a mãe expusesse um outro método visível que invocasse esse atributo ou método privado.

SUPER E SUB CLASSE

A nomenclatura mais encontrada é que `Funcionario` é a **superclasse** de `Gerente`, e `Gerente` é a **subclasse** de `Funcionario`. Dizemos também que todo `Gerente` **é um** `Funcionario`. Outra forma é dizer que `Funcionario` é a classe **mãe** de `Gerente`, e `Gerente` é a classe **filha** de `Funcionario`.

E se precisamos acessar os atributos que herdamos? Não gostaríamos de deixar os atributos de `Funcionario`, `public`, pois, dessa maneira, qualquer um poderia alterar os atributos dos objetos desse tipo. Existe um outro modificador de acesso, o `protected`, o qual fica entre o `private` e o `public`. Um atributo `protected` só pode ser acessado (visível) pela própria classe, suas subclasses e classes encontradas no mesmo pacote.

```
public class Funcionario {
    protected String nome;
    protected String cpf;
    protected double salario;
    // métodos devem vir aqui
}
```